

4.1/4

Technical Defence Report 2

Queen Amann

**Clémence GOUTAUDIER - Matteo PARNIERE-BOSQUEZ -
Yasmine DGHOUGHY - Céleste LECAMP - Mary Bruny
NGUEDIA NGUIAZON
17/03/2026**

Table of contents

1. Introduction.....	2
1.1 Context of the project	2
1.2 Objectives of the second technical defence.....	2
2. Project Overview and Technical Requirements	2
2.1 New implementations	2
2.2 Technologies and tools used	2
3. Team Organization and Task Distribution	3
3.1 Team presentation	3
3.2 Task distribution and responsibilities	5
4. Level Designing	6
4.1 A1 Level “Earth”	6
4.2 A2 Level “Air”.....	7
4.3 B1 Level “On Fire”	7
4.4 B2 Level “Water”	8
5. Project Advancement and Proof of Concept.....	8
5.1 Global project status	8
5.2 Tutorial implementation (Proof of Concept)	9
5.2.1 Player Movement and Controls.....	9
5.2.2 Combat Mechanics.....	9
6. Technical Implementations	10
6.1 Multiplayer and networking	10
6.2 Combat System and Health Management.....	11
6.2.1 Player health and Damage system.....	11
6.2.2 Enemy Health and Defeat Logic	12
7. User Interface and Menus	13
8. Visuals and Audio	13
9. Website Development	14
10. Planning and future work.....	15
11. What we learnt	17
12. Conclusion	18



1. Introduction

1.1 Context of the project

This project is part of a collaborative academic assignment focused on the design and development of a video game. The objective is to apply theoretical and technical knowledge acquired during the curriculum to a concrete, multidisciplinary project involving programming, game design, artificial intelligence, networking, visual design, and project management. The project is developed as a team, combining research, experimentation, and implementation. Attention is given to collaboration, technical feasibility, and creative originality. In addition to the game itself, the project also includes the development of a website and technical documentation, ensuring a complete presentation of both the creative and technical aspects of the work.

1.2 Objectives of the second technical defence

The second technical defence aims to present the current state of the project and demonstrate the progress achieved during the second development phase. Its main objectives are to present the advancement of the project since the last report, explain the problems we encountered during this second phase and how we adapted in order to continue working in the best conditions to finish all the tasks we assigned for this period. To do so we will demonstrate a proof of concept, including newly implemented gameplay mechanics. We will also showcase the second technical implementations, such as multiplayer networking and artificial intelligence. We will then present the project website and its role within the project and outline the planning and objectives for the next development stages. This defence focuses on validating the technical foundations of the project and showcasing new implementations. It highlights the learning process, the challenges encountered, and the solutions implemented, while establishing a clear roadmap for final improvements.

2. Project Overview and Technical Requirements

2.1 New implementations

A notable addition to our game is the online multiplayer aspect. Two players can simultaneously play the game on the same server. We have also added the combat system, which finally lets players and enemy entities interact with one another through attack and damage. Finally, our players are now able to wall-jump, which offers a whole new dimension to our game. On the website side, it is now responsive!

2.2 Technologies and tools used

The game was developed using Python as the main programming language, chosen for its readability and flexibility. We were encouraged as well to use the PyGame library for game development, including rendering, input handling, basic physics, and game logic.

Visual Studio Code and *PyCharm* were used as the main development environments, providing debugging tools and workflow flexibility during implementation. Other libraries were used, such as



sockets, threading, JSON, and sys. To ensure clean and coherent work we used GitHub, which we also used for the host of the website.

As for communication, we use *WhatsApp* to discuss quickly and *Discord* for more detailed and complex questions and problems. *Notion* is used to gather all the information we need and the project's advancement.

To simplify level designing, Matteo coded a level editor just for the Queen Amann Game. Thanks to this level editor, we can easily create maps by placing blocks (with different textures depending on the look we search). The map is in JSON format. We can place and delete any texture and it is designed to avoid coding every texture by hand. We can choose to place the texture on the 32x32 grid, or not. This allows the user to place textures very efficiently. Some textures include collisions (such as the ground), and some do not (like houses and other decorative elements). This tool, coded by one of our members will be especially valuable when we will create the distinct levels.



(Personal 4.¼ level editor with a houses, natural decoration and ground)

3. Team Organization and Task Distribution

3.1 Team presentation

Our company is called 4.¼ (Quatre-Quarts). This name reflects both our team spirit and our way of working: like a recipe, each member plays an essential role to create a balanced and coherent project. Quatre Quarts was founded in September 2025 as part of our academic program. We had to create groups to develop a video game, and we naturally chose to work together. We already knew each other, got along well, and were curious to see how far we could go as a team. In October, we welcomed a new member, and today we are five, a figure of which we are immensely proud. The core values of Quatre Quarts are creativity, both artistic and technical, and curiosity. We love understanding how things work, experimenting on our own, and learning by doing. Even though this is our first video game project, both individually and collectively, we see it as a strength. Everything is new, everything is to be learned,



tested, and built together. This project is a true learning adventure for us. It allows us to discover teamwork, Python development, the use of PyGame, and the various stages of a technical project including its successes and challenges. Our goal is not only to create a video game, but to grow through this experience: gaining new skills, strengthening our ability to work together, and developing a clearer understanding of the world of computer science.

Each member of Quatre Quarts has a key role:

Clémence GOUTAUDIER oversees the technical side of the project. She manages server creation and coordinates the code, ensuring that all technical components work smoothly together. She plays a key role in maintaining the stability and reliability of the project and makes sure that technical issues are identified and resolved efficiently.

Matteo PARNIERE-BOSQUEZ oversees the artistic and visual aspects of the game. He works on illustrations, the visual identity of the company, making sure that the artistic direction remains consistent. On top of that, he contributes to the sound design and has already composed several original music tracks, which play a significant role in creating the game's atmosphere and enhancing player immersion. He also created the player's movement code and a level editor.

Yasmine DGHOUGH supervises the coordination and overall project follow-up. She makes sure the project moves forward smoothly by maintaining excellent communication within the team and ensuring that everyone stays involved. She also helps keep a global vision of the project, balancing creativity and organization while keeping the team focused on the main objectives. She also created some parts of our AI such as wandering AI.

Céleste LECAMP is responsible for scheduling and the website. As Scheduling Manager, she ensures that deadlines are respected and that the project progresses according to the planned timeline. As Website Director, she designs and manages the website, focusing on the interface to help users discover and understand the game.

Mary Bruny NGUEDIA NGUIAZON brings strong knowledge in video game development to the team. She provides valuable guidance on the technical direction of the project, including gameplay systems and the user interface. She is also eager to learn and enjoys exploring new technical challenges throughout the development process. She worked on the enemy's AI.



(Drawing of the group members by Matteo Parniere-Bosquez)



3.2 Task distribution and responsibilities

Tasks	Manager	Alternate
Game Executable	Céleste	Clémence
User interface	Mary	Matteo
Game Basics	Matteo	Mary
AI	Mary/Yasmine	
Game test	Yasmine	Clémence
Story & Level Designs	Everyone	Everyone
Server & Multiplayer	Clémence	Céleste
Website	Céleste	Matteo
Sound & Graphism	Matteo	Yasmine

(table of the task distribution)

Clémence GOUTAUDIER works on the server and the multiplayer aspect of the game. She wrote the Technical Defence Report. She also designed level *B1* centered around Fire. She will make sure to merge correctly the game play mechanics with the server, to make the game smooth and fluid.

Matteo PARNIERE-BOSQUEZ Created the tutorial (storyboard and visual ideation of the tutorial). He also worked on the game basics such as player movement and collisions. He drew the sceneries and characters as he oversees the artistic and visual aspects of the game, as well as the player mechanics aspect. He will oversee adding the combat system to the now multiplayer game.

Yasmine DGHOUGH worked on the wandering AI, on the UI (User Interface) and on the *level A2* centred around Air.

Céleste LECAMP worked on the design of the website and works on the development of it, ensures the project's timeline is respected and creates all the PowerPoints for the different defences. She also designed the *level A1* centred around earth.

Mary Bruny NGUEDIA NGUIAZON worked on enemies' AI, on the tutorial (storyline and gameplay mechanics) and designed level *B2* centred around water.

The "level A-B 1-2" are four levels of the game Queen Amann that would each have a different mechanism, storyline and design.

We expected for the first defence to have a functional server as well as a website and some basic game mechanics (such as movement and collisions). We had a lan server, a website for desktop and some of the basic movements.

For the second defence, we expected to have all the level designing, to switch to an online server and have a functional multiplayer game, a satisfying enemy artificial intelligence and successfully implement all the game basics.



4. Level Designing

4.1 A1 Level “Earth”

The design of the Earth level was inspired by platformer environments, in particular vertical exploration. The reference image shows a landscape made of floating earth platforms covered with grass, creating a layered environment where players must navigate through jumps and narrow passages.

For this level, our main source of inspiration comes from the structure of ant colonies. Ant nests are complex underground systems made of interconnected tunnels and chambers that extend vertically and horizontally through the ground. This natural structure influenced the way the platforms and paths are organized in the level.

Like an ant nest, the level is designed with multiple passages and vertical routes, encouraging players to move up and down through the terrain. The narrow spaces between platforms resemble underground tunnels, creating a feeling of navigating through the interior of the earth.

This inspiration also helps reinforce the Earth theme by representing the ground not only as a surface but as a living environment with depth and hidden paths. The idea was to recreate the sensation of exploring a natural underground structure, where each platform can be seen as part of a larger network like the chambers and tunnels found in ant colonies.



(Personal 4.¼ draft of the first level)



4.2 A2 Level “Air”

This level is designed around the theme of air, where wind plays a central role in the player’s movement and progression. The player encounters areas with upward air currents that allow them to rise and reach higher platforms, creating a feeling of lightness and vertical exploration. The platforms are made of clouds, some stable and others moving left to right, encouraging careful movement and anticipation.

To increase difficulty, moving hurricane-like gusts pass through certain sections of the level, forcing the player to time their jumps precisely to avoid being pushed away or falling. This combination of vertical wind mechanics, unstable cloud platforms, and timed hazards creates a dynamic and engaging gameplay experience centred on control, timing, and adaptation.



(design of a cloud made by Matteo)

4.3 B1 Level “On Fire”

The B1 level, centred around the theme of fire, is designed to introduce environmental mechanics based on the interaction between dissimilar materials and the unique abilities of each playable character.

One of the main design ideas behind this level is the concept that different characters interact differently with certain materials present in the environment. In particular, the character Tonatiuh, who appears as a skeleton surrounded by flames, possesses the ability to safely interact with fire-based elements. As a result, Tonatiuh can walk through areas containing flames or activate mechanisms located within hazardous fire zones without taking damage.

The second character, Citlalli, does not share this ability. This difference creates situations in which players must cooperate to overcome environmental challenges. The environment of this level is inspired by desert landscapes associated with Latin American regions. The level therefore includes visual elements such as sand, desert terrain, temple structures, and vegetation such as vines integrated into the ruins. These decorative elements contribute to creating a distinctive visual identity for the level while remaining consistent with the artistic direction of the game.

In addition to fire hazards, the level also introduces quicksand areas. These zones act as environmental traps that slow down the player’s movement or gradually pull the character downward if the player remains stationary for too long. To escape these areas safely, players must move quickly and carefully across the unstable ground.



4.4 B2 Level “Water”

This is the last level of the game which is based on a watery world; After a long quest in the other world, the player arrives at the end of their quest and is looking for the last ingredient for the preparation of the Kouign amann to save the queen, for that the player must locate and acquire the “Mist sugar” also called Foam frost. The foam frost is a crystallized foam that forms on cold waterfalls, looks like sugar crystals and is perfect for topping, protected by the waterfall spirit called “Nereid Tears”.

The level would mostly consist of waterfalls, providing a watery world environment this implies the introduction of some new characters, here the enemy would no more be a chocolate bars but crabs that try to prevent the player from attaining its objective of collection, which is from entering a cave which is hidden behind big waterfalls. Inside that cave would be first guarded by the water serpent which is a mini-boss, and finally the big boss (Nereid Tears).

The change in the enemy’s character is due to a change in the environment, this environment would encourage upward movement by player to move from one slippery block to another, and the presence of a mini boss and boss is a promise of an entertaining action there by capturing the player's attention and retaining them in the game.

The skills such as jumping, wall jumping, learning how to use of the player’s controls and fighting that the player has obtained throughout his quest would enable him to survive through this final level, so this level would be defined in such a way that anyone which has not been able to go through the other levels would not be able to survive this one.

5. Project Advancement and Proof of Concept

5.1 Global project status

Since the first technical defence, the project has progressed from a conceptual prototype to a functional multiplayer game prototype. Several fundamental systems have now been implemented, allowing players to connect to a server, control their characters, interact with enemies, and explore a playable environment.

The primary objective during this phase of development was to implement the core technical foundations of the game before focusing on additional gameplay features. These foundations include player movement, physics simulation, collision detection, multiplayer synchronization, enemy artificial intelligence, and the development of internal tools to simplify level creation.

The current version of the game can be launched and played by multiple users simultaneously. When starting the game, players are prompted to enter a display name. This name is then transmitted to the server and displayed above the player character during gameplay. This feature allows players to easily identify each other during multiplayer sessions.

The multiplayer system allows two players to connect to the same server and interact in the same game environment in real time. Player movements, actions, and position updates are synchronized across clients, ensuring that both players see the same player behaviour on their respective screens.



Several core gameplay mechanics have been successfully implemented, including:

- Horizontal player movement
- Gravity and vertical physics simulation
- Jump and double jump mechanics
- Wall jumping mechanics
- Collision detection with terrain blocks
- Camera tracking system following the player
- Enemy interaction and combat system
- Player health system and damage feedback

Although the game currently contains a single functional level, the internal level editor developed by the team now allows new environments to be created quickly. This tool significantly accelerates level design and makes future content creation much easier.

5.2 Tutorial implementation (Proof of Concept)

A tutorial level was implemented early in the development process to introduce players to the basic mechanics of the game.

The purpose of the tutorial is to familiarize players with the controls and the general behaviour of the character within the game environment. By introducing mechanics gradually, the tutorial helps players understand how to move, jump, and interact with the world.

This level also serves as a technical demonstration that the different systems implemented by the team can function together within a coherent gameplay environment.

5.2.1 Player Movement and Controls

Player movement mechanics are fully implemented and form the foundation of the gameplay experience.

The controls are intentionally simple to ensure that the game remains accessible to new players. Movement is performed using directional keys that allow the player to move horizontally across the environment. Gravity is constantly applied to the player character, causing the character to fall when not standing on a platform. Jumping temporarily counteracts this force and allows the player to reach higher platforms.

The movement system also includes a double jump mechanic, allowing the player to perform a second jump while already in the air. This ability improves manoeuvrability and allows players to navigate more complex platforming sections. Another mechanic that has been implemented is wall jumping, which allows the player to push away from vertical surfaces. This mechanic adds an additional layer of mobility and encourages creative movement through the environment.

To prevent unintended gameplay exploits, the double jump cannot be used immediately after a wall jump.

5.2.2 Combat Mechanics

In addition to movement, a basic combat system has been introduced to create interactions between the player and enemies. The player can perform an attack action which triggers an attack animation and generates an attack zone in front of the character. Enemies located within this zone can receive damage. Combat currently focuses on simple interactions where the player must avoid enemy attacks while



attempting to defeat them. Enemies can attack the player as well, but their attacks are regulated by a cooldown system. This system prevents enemies from attacking continuously and ensures that players have enough time to react.

6. Technical Implementations

6.1 Multiplayer and networking

One of the most technically challenging aspects of the project has been the implementation of a multiplayer system capable of synchronizing player actions in real time. The multiplayer architecture is based on a client-server model, where all players connect to a centralized server responsible for coordinating the game state.

The server is implemented in Python and deployed online using the Koyeb hosting platform, allowing players to connect remotely via the internet. The server acts as an intermediary between all connected clients, receiving gameplay data from each player and redistributing it to the other players.

When a player performs an action (such as moving, jumping, or attacking), the client sends the relevant information to the server. The server then processes this data and broadcasts it to all connected clients. Each client updates its local representation of the other players based on the received information.

This architecture ensures that player movements appear synchronized across all connected machines.

Currently, the following elements are synchronized between players:

- Player position
- Player actions (movement, jumping, attacking)
- Player display name

This synchronization allows players to observe each other's movements in real time during gameplay.

Some elements of the game logic still operate locally. Enemy behaviour is currently computed locally on each client. While enemies can detect and interact with players on the local machine, their behaviour is not yet fully synchronized across all clients. This means that each player may see slightly different enemy reactions depending on their local game state.

Two potential solutions are currently being considered for future development:

- 1) Server-controlled enemies, where enemy actions would be computed on the server and broadcast to all players.
- 2) Client-local enemies with remote detection, where enemies remain locally simulated but can detect all connected players and reacting accordingly.

Both approaches present advantages and technical challenges, and further experimentation will determine the most efficient solution.



Despite these limitations, the current multiplayer implementation already demonstrates stable real-time player interaction and provides a functional foundation for cooperative gameplay.



(Screenshot of the multiplayer display)

6.2 Combat System and Health Management

To make the gameplay more interactive and give the player the ability to defend themselves, a simple combat system was implemented. The objective of this system is to allow the player to attack enemies, take damage when attacked, and manage health through a life bar. This adds a new layer of interaction between the player and enemies and makes encounters more meaningful within the game.

The long-term goal of this system is to create a balanced combat loop where the player can react to enemy attacks, avoid damage, and defeat enemies through well-timed actions. To achieve this, the system manages player health, enemy health, attack detection, and temporary invulnerability after taking damage.

To organize these mechanics, the combat system is divided into several components.

6.2.1 Player health and Damage system

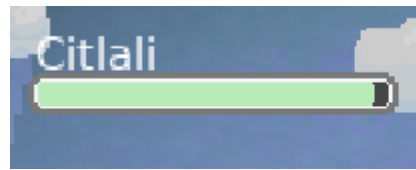
A health system was added to the player to represent the character's endurance during gameplay. Each player starts with a maximum health value and loses health when receiving damage from enemies.

To prevent the player from losing health too quickly when staying close to an enemy, a short invulnerability period was introduced after each hit. During this period, additional damage is ignored. This mechanism ensures that damage occurs in controlled intervals rather than continuously at every frame.

If the player's health reaches zero, the character is automatically respawned at the initial spawn point of the level and their health is restored. This allows the player to continue playing without restarting the entire game.



The health information is also displayed visually through a life bar, which changes colour depending on the remaining health. This visual feedback helps the player quickly understand their current situation during gameplay.



(Health bar from the game)

In addition to receiving damage, the player can now attack enemies. When the attack command is triggered, the player enters an attack state and plays an attack animation.

To determine whether an enemy is hit, the system generates an attack zone positioned in front of the player. This zone represents the effective range of the attack and depends on the direction the player is facing. Only enemies located inside this area can receive damage.

To maintain fair gameplay, each attack animation can damage only one enemy and only once per animation cycle. This prevents a single attack from repeatedly damaging the same enemy during multiple frames.

The system checks which enemy is the closest within the attack area and applies damage to that enemy. This approach ensures that attacks feel consistent and that the player hits the intended target.

An additional rule was implemented to handle situations where the player falls outside the playable area. The objective of this feature is to ensure that the player cannot continue indefinitely below the level and to maintain a consistent gameplay flow.

At each frame, the system checks the vertical position of the player. If the player's position exceeds a predefined threshold (meaning they have fallen too far below the map), the game considers this as a death condition.

When this condition is triggered, the player is automatically reset to their initial spawn point. Their position, velocity, and health are restored to their default values. The camera is also repositioned accordingly to follow the player from the starting location.

This mechanism acts as a safety system that prevents the player from getting stuck outside the level and ensures that gameplay can continue smoothly after a fall.

6.2.2 Enemy Health and Defeat Logic

Enemies were also given their own health values so that they can react to the player's attacks. Each enemy starts with a small amount of health and loses health whenever the player successfully hits them.

When an enemy's health reaches zero, the enemy is considered defeated and disappears from the level. This simple system allows the player to eliminate threats and progress more safely through the environment.

To make combat interactions easier to detect and feel more natural, enemies use a slightly larger target area than their visual sprite. This adjustment compensates for the spacing that sometimes exists in sprite graphics and ensures that attacks visually aligned with the enemy still register correctly.



7. User Interface and Menus

The user interface plays an important role in helping players understand the state of the game and interact with its different systems.

At the current stage of development, the interface focuses mainly on providing essential information to the player during gameplay.

One of the most important interface elements is the display of the player's name above the character. Since the game is designed as a multiplayer experience, this feature allows players to easily identify each other while playing together.

Another important element is the health bar, which visually represents the remaining health of the player. The life bar provides immediate feedback during combat situations and helps players assess their current condition.

8. Visuals and Audio

In parallel with the technical development of the game, work has also been carried out on the visual and audio aspects of the project.

The visual identity of the game is created primarily through character sprites, environmental textures, and background elements. These graphical assets contribute to the atmosphere of the game world and help define the aesthetic style of the project.

The player characters have been designed and animated using digital drawing tools, such as Procreate. Several animation states have been created to represent the different actions performed by the characters during gameplay. These include idle animations, running animations, jumping animations, and attack animations.



(in game sprites of the characters and sprite of an enemy chocolate bar attacking)

Environmental elements such as terrain blocks, decorative objects like houses, and natural elements (flowers, grass etc.) are also progressively being added to the game environments. These elements help create visually distinct levels and make the world feel more alive and immersive. Future environments will include additional decorative elements such as temples, vegetation, water features, and architectural structures inspired by the artistic direction of the game.



(in game decoration)

9. Website Development

The website is currently online and hosted on GitHub Pages. It is fully functional and accessible to users.

At its current stage, the website provides:

- Information about the project and team members
- A presentation of the game's story and narrative universe
- Access to downloadable documents such as:
 - Technical Specifications Document
 - Functional Specifications Document
 - Technical report 1
 - Technical Presentation Plan 1
 - Technical report 2
 - Technical Presentation Plan 2
- A contact email, allowing users or visitors to get in touch with the team

The website has been improved with several new features to make it more user-friendly and accessible on all devices. It is now fully responsive, working well on phones, tablets, and desktop computers. The Gameplay and Graphics page has been updated with new content, including short gameplay videos. These additions give visitors a brief but engaging glimpse of the game's style, characters, and mechanics. The "back to top" feature has also been adjusted to scroll more slowly, creating a smoother and more comfortable experience for users navigating longer pages. Overall, these updates enhance both the functionality and the visual appeal of the website, while laying the groundwork for future content, such as additional gameplay footage.



10. Planning and future work

Although a large part of the core mechanics has now been implemented, the final phase of development will focus on completing and stabilizing the full gameplay loop through the integration of level-specific mechanics and systems.

The current version of the game already allows players to connect to the server, move within the environment, interact with enemies, and use basic combat mechanics.

The gameplay loop will therefore be finalized as follows:

- Enter a level with specific environmental mechanics
- Interact with level-specific hazards and systems
- Progress through platforming and cooperative challenges
- Face increasingly complex enemy encounters
- Reach the end of the level and unlock the next area

Rather than introducing new conceptual features, this phase focuses on the technical implementation of environmental systems directly derived from the level designs.

Each level will include dedicated mechanics implemented at the code level, such as:

- Damage zones: areas (e.g. fire, hazardous terrain) applying continuous or periodic damage when a player enters a collision zone
- Material-based interactions: conditional logic allowing certain characters to ignore or interact differently with specific elements (e.g. fire resistance for one character)
- Movement modifiers: terrain affecting player physics, such as reduced speed (quicksand), forced vertical movement (air currents), or low-friction surfaces (water/slippery blocks)
- Trigger-based systems: buttons, levers, or activation zones modifying the environment (platform spawning, path opening, hazard deactivation)

These systems will be implemented using collision detection, state management, and entity interaction logic, ensuring that each level introduces distinct gameplay constraints while remaining consistent with the core engine.

Enemy systems will also be extended technically. While the current version includes a single enemy type with basic detection and attack cooldowns, additional behaviours will be implemented, such as:

- Area pressure mechanics, forcing player movement (e.g. enemy zones)
- Spawn systems for multiple enemies triggered by player position
- Boss entities combining multiple attack patterns and higher health pools

The multiplayer system, which already supports real-time synchronization of player positions and actions, will be extended with additional structural features:

- Implementation of a room-based session system allowing players to join specific instances



- Integration of a menu interface for level selection and session management
- Optional implementation of persistent data storage (usernames, progress)
- Improvements in data exchange optimization between client and server

In parallel, the user interface system will be completed. This includes the implementation of navigable menus (main menu, room selection, level selection) and the addition of transition effects between game states. These transitions will be handled through screen overlays and timed rendering effects to ensure visual continuity.

In addition to gameplay and networking systems, a significant part of the final development phase will focus on the implementation of a complete and functional User Interface (UI). Unlike the current version, where interaction is minimal and mostly limited to gameplay display elements, the final version of the game will include a structured interface system allowing full navigation and configuration by the player.

From a technical perspective, the UI will rely on event-driven input handling, where user interactions (keyboard or mouse inputs) trigger transitions between interface states. Each menu will be managed through a structured system of buttons and selectable elements, with clear navigation logic.

Special attention will be given to the integration between UI and gameplay systems. For example, player names entered in the menu will be transmitted to the server and displayed in-game, and selected levels will directly determine which environment is loaded.

Additionally, visual transitions between UI states will be implemented to improve user experience. These transitions will be handled through rendering effects, such as screen fades or circular closing animations, ensuring continuity between menus and gameplay.

Regarding audio, the final stage will focus on full integration of sound systems. This includes:

- Associating sound effects with player actions (jump, attack, damage)
- Adding audio feedback to enemy behaviours and environmental interactions
- Integrating background music dynamically depending on the level

All audio events will be triggered programmatically based on in-game events to maintain synchronization with gameplay.

Finally, one of the most critical tasks in this phase is the complete integration of all subsystems into a stable version of the game. Although a first merge has already been achieved, further refinement is required to ensure consistency between gameplay logic, multiplayer synchronization, and rendering systems.

This final integration phase will focus on:

- Debugging cross-system interactions
- Ensuring consistency between local and networked behaviours
- Optimizing performance and stability
- Finalizing a fully playable version of the game



This stage represents the transition from a functional prototype to a complete and coherent game experience, ready for final presentation.

In preparation for the final defence, the project will also include the production of all required deliverables associated with a complete and exploitable application. These elements go beyond the development of the game itself and aim to ensure that the project can be understood, installed, and used independently.

The team will therefore prepare a structured presentation plan for the final defence, allowing for a clear and organized demonstration of the project. A finalized project report will also be produced, including all technical explanations, design choices, and development progress.

In addition, an operating manual will be created to describe how the application can be used, maintained, and potentially extended. This document will focus on practical aspects of the system, including its structure, usage conditions,

and technical requirements.

The project will be delivered as a fully packaged version on a USB drive, ensuring that the game can be executed in a stable and accessible manner. This packaged version will include all necessary files and dependencies required for proper execution.

A website presenting the project will also be finalized, serving as a platform for communication, documentation, and visual presentation of the work accomplished.

Finally, a complete installation procedure will be written. This document will provide step-by-step instructions allowing any user to install, configure, and launch the game, including both client and server components if necessary.

11. What we learnt

During this second phase of the project, the team gained valuable experience both from a technical and an organizational perspective. One of the most significant lessons learned concerns the complexity of integrating different parts of a project developed collaboratively.

At this stage of development, we were able to successfully merge the different blocks of code produced by each team member. However, this process proved to be much more challenging than initially expected. These difficulties mainly arose from the way the project was structured. Since the work was divided among team members, each person developed a specific part of the system independently, such as the player mechanics, enemy behaviour, or the server.

As a result, each component was built using different methodologies, coding styles, and architectural approaches. While this division of labour allowed for faster individual progress, it introduced significant complexity when attempting to combine all components into a single, coherent system. The integration process required a deep understanding of each section of the code, as well as the ability to identify how these independent systems could interact correctly. This often involved modifying existing code, resolving incompatibilities, and debugging unexpected issues.

Another major lesson learned during this phase concerns the importance of clear written communication within the code itself. While verbal explanations between team members can be helpful in the short term, they are not sufficient when working on a shared codebase over a longer period.



We realized that properly structuring the code and adding clear comments is essential. Comments allow developers to quickly understand the role of different sections of the code without having to analyse every line in detail. By organizing the code into well-defined sections and documenting the purpose of functions and systems, it becomes significantly easier for other team members to navigate and understand the project.

This is particularly important when working with code written by someone else. Since each developer has a different programming style and logic, reading another person's code can be difficult and time-consuming without proper explanations. Clear comments act as guidance and reduce the effort required to understand how a system works.

In addition to code documentation, we also learned the importance of maintaining clear and precise commit messages when using version control tools such as GitHub. While making frequent commits is good practice, the quality of the commit messages is equally important.

During development, we encountered situations where unclear commit messages made it difficult to track changes. For example, some commits were labelled with non-descriptive names such as a single letter. In such cases, it becomes nearly impossible to understand what modifications were made, especially when revisiting the project later or debugging an issue.

This issue became particularly evident during the development of the server, where many small changes were made through trial and error. In one instance, dozens of commits were created, each corresponding to minor modifications. Without clear descriptions, it was extremely difficult to identify what had changed between versions or to understand why a particular version was functioning correctly or not.

This experience demonstrated that even small changes should be documented clearly in commit messages. A well-written commit message should describe what was modified and, when relevant, why the modification was made. This practice significantly improves the ability to track the evolution of the project and simplifies debugging and collaboration.

By applying these lessons, we aim to improve the efficiency of our development process and reduce integration issues in the remaining stages of the project.

12. Conclusion

This second technical defence presents the progress achieved during the second phase of development of the Queen Amann project. The team has successfully implemented several essential systems, including multiplayer networking, player movement mechanics, enemy interactions, and the basic combat system. These elements demonstrate that the technical foundations of the game are now functional.

Although the game is still under development, the current prototype already provides a clear demonstration of the intended gameplay experience. Players can connect to the same server, explore the environment, interact with enemies, and cooperate within the game world.

Several improvements are still planned for the next phase of development, including additional gameplay mechanics, improved enemy behaviours, expanded multiplayer features, and the integration of additional visual and audio elements. The team will now focus on refining the existing systems, integrating the remaining components, and completing the game experience before the final project submission.

